

Network Packet Capture Analysis

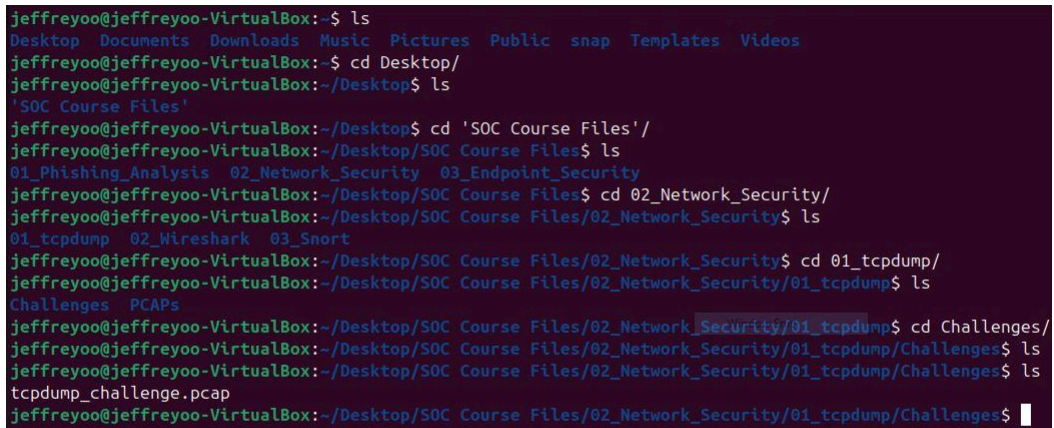
with **tcpdump**

Jeffrey Obi
May 2026

Network Packet Capture Analysis with tcpdump

Environment Setup in the Terminal: Before beginning, navigate to the directory containing your PCAP file to keep your commands clean and easy to read. Open your Ubuntu terminal and run: `cd`

`~/Desktop/02_Network_Security/01_tcpdump/Challenges/`



```
jeffreyoo@jeffreyoo-VirtualBox:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
jeffreyoo@jeffreyoo-VirtualBox:~$ cd Desktop/
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop$ ls
'SOC Course Files'
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop$ cd 'SOC Course Files'/
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files$ ls
01_Phishing_Analysis  02_Network_Security  03_Endpoint_Security
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files$ cd 02_Network_Security/
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security$ ls
01_tcpdump  02_Wireshark  03_Snort
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security$ cd 01_tcpdump/
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump$ ls
Challenges  PCAPs
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump$ cd Challenges/
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ ls
tcpdump_challenge.pcap
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$
```

Figure 1: Ubuntu terminal showing the change of directory to the tcpdump challenges folder.

1. How many total packets are in the tcpdump_challenge.pcap packet capture?

Command:

`tcpdump -n -r tcpdump_challenge.pcap --count`

Command Breakdown:

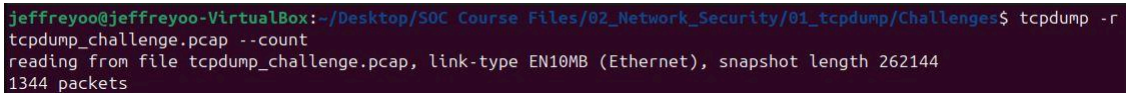
- `tcpdump`: Invokes the packet analyzer.
- `-n`: Disables DNS resolution (prevents converting IP addresses to hostnames), which speeds up the read process significantly.
- `-r tcpdump_challenge.pcap`: Reads the packets from the specified file instead of a live network interface.
- `--count`: Counts the total number of packets in the file without displaying them all.

Terminal Return:

You will see a single integer (e.g., **4582**). Since tcpdump outputs one line per packet by default, this number represents the total packet count.

Answer:

1344

A terminal window screenshot showing the command 'tcpdump -r tcpdump_challenge.pcap --count' being executed. The output displays the file path, link type (EN10MB Ethernet), snapshot length (262144), and the total packet count (1344 packets).

```
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -r
tcpdump_challenge.pcap --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
1344 packets
```

Figure 2: Terminal output showing a total of 1344 packets in the capture file.

2. How many ICMP packets are in the challenge.pcap packet capture?

Command:

```
tcpdump -n -r tcpdump_challenge.pcap icmp --count
```

Command Breakdown:

- **tcpdump**: Invokes the packet analyzer.
- **-n**: Disables DNS resolution (prevents converting IP addresses to hostnames), which speeds up the read process significantly.
- **-r tcpdump_challenge.pcap**: Reads the packets from the specified file instead of a live network interface.
- **icmp**: This is a BPF (Berkeley Packet Filter) syntax that tells tcpdump to *only* display packets utilizing the ICMP protocol.
- **--count**: Counts the total number of packets in the filtered output without displaying them all.

Terminal Return:

You will receive an integer representing the exact number of ICMP (ping) packets.

Answer:

132

```
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -r tcpdump_challenge.pcap icmp --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
132 packets
```

Figure 3: Terminal output filtering for ICMP packets with a result of 132.

3. What is the ASN of the destination IP address that the endpoint was pinging?

This question is derived by a 2-step process.

Step 1

Command:

Find the destination IP address being pinged.

```
tcpdump -n -r tcpdump_challenge.pcap icmp
```

Command Breakdown:

- **tcpdump**: Invokes the packet analyzer.
- **-n**: Disables DNS resolution (prevents converting IP addresses to hostnames), which speeds up the read process significantly.
- **-r tcpdump_challenge.pcap**: Reads the packets from the specified file instead of a live network interface.
- **icmp**: This is a BPF (Berkeley Packet Filter) syntax that tells tcpdump to *only* display packets utilizing the ICMP protocol.

Terminal Return:

The command will output lines displayed in the following 2 formats:

```
IP <Source_IP> > <Destination_IP>: ICMP echo request...
IP <Source_IP> > <Destination_IP>: ICMP echo reply...
```

Note the Destination IP address on the **request** lines (**172.67.72.15**)

Step 2

Look up the ASN (Autonomous System Number) for that IP.

Command:

```
whois -h whois.cymru.com -- -v 172.67.72.15
```

Command Breakdown:

- **whois**: The primary command-line utility used to query databases that store the registered users or assignees of an Internet resource.
- **-h whois.cymru.com**: Specifies the specific WHOIS server to query; in this case, Team Cymru's specialized IP-to-ASN mapping service.
- **--**: Acts as an argument separator that tells the local whois client to stop processing options and pass all subsequent flags directly to the remote server.
- **-v**: The verbose flag requested from the Cymru server to provide descriptive headers for the output data (AS, IP, BGP Prefix, CC, Registry, Allocated, and AS Name).
- **172.67.72.15**: The specific IP address being investigated to find its associated Autonomous System Number (ASN) and registration details.

Terminal Return:

The command returns multiple details about the IP address. The first observed return on the left, is the ASN. The format is “AS” followed by a 5-digit number.

Answer:

AS13335

```
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -r
tcpdump_challenge.pcap icmp -c 5
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:53:51.081222 IP 10.0.2.10 > 172.67.72.15: ICMP echo request, id 3940, seq 1, length 64
15:53:51.108514 IP 172.67.72.15 > 10.0.2.10: ICMP echo reply, id 3940, seq 1, length 64
15:53:52.081541 IP 10.0.2.10 > 172.67.72.15: ICMP echo request, id 3940, seq 2, length 64
15:53:52.100611 IP 172.67.72.15 > 10.0.2.10: ICMP echo reply, id 3940, seq 2, length 64
15:53:53.081802 IP 10.0.2.10 > 172.67.72.15: ICMP echo request, id 3940, seq 3, length 64
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ whois -h wh
ois.cymru.com -- -v 172.67.72.15
AS      | IP      | BGP Prefix | CC | Registry | Allocated | AS Name
13335   | 172.67.72.15 | 172.67.64.0/20 | US | arin      | 2015-02-25 | CLOUDFLARENET - Cloudflare, Inc.
, US
```

Figure 4: WHOIS lookup results for IP 172.67.72.15 showing ASN 13335.

4. How many HTTP POST requests were made in the packet capture?

Command:

```
tcpdump -A -r tcpdump_challenge.pcap port 80 | grep -E POST
--count
```

Command Breakdown:

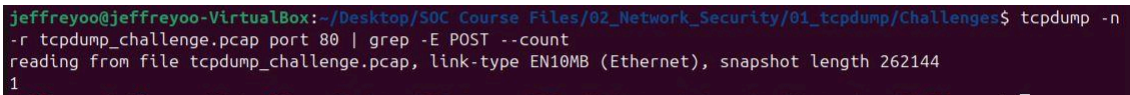
- `tcpdump`: Invokes the packet analyzer.
- `-A`: Prints the payload of the packets in ASCII (human readable plain text). This is crucial for reading plain-text application layer data like HTTP methods.
- `-r tcpdump_challenge.pcap`: Reads the packets from the specified file instead of a live network interface.
- `port 80`: Filters for standard **HTTP** traffic.
- `|`: Pipes the output from `tcpdump` into the next (filter) command.
- `grep -E POST --count`: Searches the ASCII output for the string 'POST', enabling extended regular expressions (`-E`) and returning the count of matches (`--count`).

Terminal Return:

An integer (1) showing how many times the HTTP POST method was used, indicating how many times data (like forms or credentials) was submitted to a server.

Answer:

1



```
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -n -r tcpdump_challenge.pcap port 80 | grep -E POST --count
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
1
```

Figure 5: Grep command output counting a single HTTP POST request.

5. Look for any credentials within the payloads of any HTTP packets, what is the password you uncover?

Command:

```
tcpdump -A -r tcpdump_challenge.pcap 'port 80' | grep -iE 'password=|pass=|pwd=|user='
```

Command Breakdown:

- `tcpdump`: Invokes the packet analyzer.
- `-A`: Prints the payload of the packets in ASCII (human readable plain text). This is crucial for reading plain-text application layer data like HTTP methods.

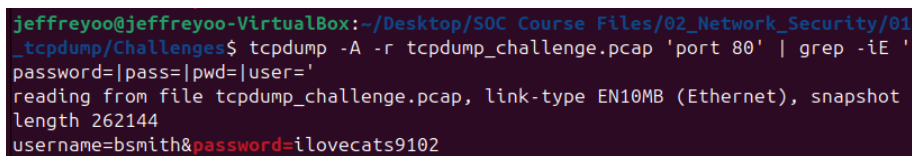
- `-r tcpdump_challenge.pcap`: Reads the packets from the specified file instead of a live network interface.
- `port 80`: Filters for standard **HTTP** traffic.
- `|`: Pipes the output from tcpdump into the next (filter) command.
- `grep -iE '...'`: Does a case-insensitive, extended regular expression search for common URL-encoded or JSON form parameters associated with logins.

Terminal Return:

Snippets of a HTTP POST payload containing the URL-encoded data `username=bsmith&password=ilovecats9102`. The password is after `"password="`.

Answer:

`ilovecats9102`



```
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -A -r tcpdump_challenge.pcap 'port 80' | grep -iE 'password=|pass=|pwd=|user='
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
username=bsmith&password=ilovecats9102
```

Figure 6: Credential extraction from the HTTP payload showing the password "ilovecats9102".

6. Aside from HTTP on port 80, what is the other most frequent well-known destination port number?

Command:

```
tcpdump -nn -r tcpdump_challenge.pcap | cut -d' ' -f 3 | cut -d'.' -f 5 | sort | uniq -c | sort -nr
```

Command Breakdown:

- `tcpdump`: Invokes the packet analyzer.
- `-nn`: Disables both hostname and port resolution (shows port numbers instead of service names, and IP addresses instead of hostnames).
- `-r tcpdump_challenge.pcap`: Reads the packets from the specified file instead of a live network interface.

- `cut -d' ' -f 3`: Extracts the destination IP and Port (the 3rd field) from the tcpdump output, using a space as the delimiter.
- `cut -d'.' -f 5`: Extracts only the destination port number from the IP.Port string, using the period as the delimiter.
- `sort | uniq -c`: Sorts the port numbers, then counts (-c) unique consecutive occurrences.
- `sort -nr`: Sorts the final port counts numerically in reverse (highest to lowest).

Terminal Return:

A list structured in left and right columns for **counts** and **port numbers**, respectively. The list is sorted by counts, so the most common port, port 80, is at the top. At count 114, the next highest port number is 44174. At count 56, the next port number is 21. Port 21 is a **well-known port**—ports usually below 1024 (e.g., 21, 22, 53, 443)—making it the correct answer.

Answer:

21

```
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -n  
-r tcpdump_challenge.pcap tcp | cut -d' ' -f 3 | cut -d'.' -f 5 | sort | uniq -c | sort -nr  
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144  
356 80  
114 44174  
56 21  
23 40124  
11 60032  
11 47582  
11 47578  
11 40122  
7 57192  
7 52366  
7 51816  
7 45562  
7 44918  
7 43030  
7 34726  
6 60086  
6 56748  
6 55574  
6 55402  
6 54584
```

Figure 7: Frequency analysis of destination ports showing port 21 as the 2nd most frequent well-known port.

7. What set of valid credentials did the endpoint use to access the file sharing server? (Format username:password)

Command:

```
tcpdump -A -r tcpdump_challenge.pcap 'port 21' | grep -i -E 'USER |PASS |230 '
```

Command Breakdown:

- `tcpdump`: Invokes the packet analyzer.
- `-A`: Prints the payload of the packets in ASCII (human readable plain text). This is crucial for reading plain-text application layer data like HTTP methods.
- `-r tcpdump_challenge.pcap`: Reads the packets from the specified file instead of a live network interface.
- `port 21`: Port 21 is the well-known port for FTP (File Transfer Protocol), the most common unencrypted file-sharing protocol seen in PCAP challenges.
- `grep -i -E 'USER |PASS |230 '`: Looks for the FTP username command, password command, and the FTP "230 User logged in" success code to verify they are *valid* credentials.

Terminal Return:

Multiple consecutive ASCII lines looking like the following:

- `USER admin;`
- `PASS password123`

The lines terminate before a login validation line containing the following:

- `User logged in.`

The “USER” and “PASS” credentials directly before the login confirmation are “demo” and “password123” respectively. This combination makes up valid credentials. The answer is a combination of the username and password in the suggested “username:password” format.

Answer:

```
demo:mypassword123
```

```

jeffreyoo@jeffreyoo-VirtualBox: ~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenges$ tcpdump -A -r tcpdump_challenge.pcap port 21 | grep -E 'u
ser|pass|login'
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:53:56.354385 IP 10.0.2.10.60032 > 194.108.117.16.ftp: Flags [P.], seq 1:13, ack 172, win 65364, length 12: FTP: USER admin
..lu....9D... ..P..TC...USER admin
15:53:56.504430 IP 194.108.117.16.ftp > 10.0.2.10.60032: Flags [P.], seq 172:208, ack 13, win 32756, length 36: FTP: 331 Password required for 'admin'.
..... ..@9D..P...A]..331 Password required for 'admin'.
15:53:57.582793 IP 10.0.2.10.60032 > 194.108.117.16.ftp: Flags [P.], seq 13:25, ack 208, win 65328, length 12: FTP: PASS admin
..lu....9D... ..dP..BC...PASS admin
15:54:00.672388 IP 10.0.2.10.47578 > 194.108.117.16.ftp: Flags [P.], seq 1:12, ack 172, win 65364, length 11: FTP: USER root
..lu....9D... ..P..TC...USER root
15:54:00.825871 IP 194.108.117.16.ftp > 10.0.2.10.47578: Flags [P.], seq 172:207, ack 12, win 32757, length 35: FTP: 331 Password required for 'root'.
..... ..P...m...331 Password required for 'root'.
15:54:02.020441 IP 10.0.2.10.47578 > 194.108.117.16.ftp: Flags [P.], seq 12:26, ack 207, win 65329, length 14: FTP: PASS password123
..lu....9D... ..P..TC...PASS password123
15:54:06.438396 IP 10.0.2.10.47582 > 194.108.117.16.ftp: Flags [P.], seq 1:21, ack 172, win 65364, length 28: FTP: USER administrator
..lu....9D... ..P..TC...USER administrator
15:54:06.571114 IP 194.108.117.16.ftp > 10.0.2.10.47582: Flags [P.], seq 172:216, ack 21, win 32748, length 44: FTP: 331 Password required for 'administrator'.
..... ..AQDP...7...331 Password required for 'administrator'.
15:54:07.940494 IP 10.0.2.10.47582 > 194.108.117.16.ftp: Flags [P.], seq 21:36, ack 216, win 65328, length 15: FTP: PASS password
..lu....9D... ..ZP...C...PASS password
15:54:12.163541 IP 10.0.2.10.40122 > 194.108.117.16.ftp: Flags [P.], seq 1:13, ack 172, win 65364, length 12: FTP: USER admin
..lu....9D... ..7.P..TC...USER admin
15:54:12.308641 IP 194.108.117.16.ftp > 10.0.2.10.40122: Flags [P.], seq 172:208, ack 13, win 32756, length 36: FTP: 331 Password required for 'admin'.
..... ..7.3.v.P...H..331 Password required for 'admin'.
15:54:13.997473 IP 10.0.2.10.40122 > 194.108.117.16.ftp: Flags [P.], seq 13:31, ack 208, win 65328, length 18: FTP: PASS password123
..lu....9D... ..7.P..BC...PASS password123
15:54:16.788257 IP 10.0.2.10.40124 > 194.108.117.16.ftp: Flags [P.], seq 1:12, ack 172, win 65364, length 11: FTP: USER demo
..lu....9D... ..CHP..TC...USER demo
15:54:16.928705 IP 194.108.117.16.ftp > 10.0.2.10.40124: Flags [P.], seq 172:248, ack 12, win 32757, length 76: FTP: 331 Anonymous login OK, send your complete
email address as your password.
..... ..CH...P...3b..331 Anonymous login OK, send your complete email address as your password.
15:54:18.648134 IP 10.0.2.10.40124 > 194.108.117.16.ftp: Flags [P.], seq 12:27, ack 248, win 65288, length 15: FTP: PASS password
..lu....9D... ..C.P...C...PASS password
15:54:18.772981 IP 194.108.117.16.ftp > 10.0.2.10.40124: Flags [P.], seq 248:276, ack 27, win 32742, length 28: FTP: 230 User 'demo' logged in.
..... ..C...P...J]..230 User 'demo' logged in.

```

Figure 8: FTP traffic capture showing a successful login for the user "demo".

8. What is the name of the file that was retrieved from the file sharing server?

Command:

```
tcpdump -A -r tcpdump_challenge.pcap 'port 21' | grep 'RETR'
```

Command Breakdown:

- **tcpdump**: Invokes the packet analyzer.
- **-A**: Prints the payload of the packets in ASCII (human readable plain text). This is crucial for reading plain-text application layer data like HTTP methods.
- **-r tcpdump_challenge.pcap**: Reads the packets from the specified file instead of a live network interface.
- **port 21**: Port 21 is the well-known port for FTP (File Transfer Protocol), the most common unencrypted file-sharing protocol seen in PCAP challenges.
- **grep 'RETR'**: In FTP, the **RETR** (Retrieve) command is issued by the client to tell the server to download a specific file.

Terminal Return:

A line containing **RETR readme.txt** The string following “RETR” is the filename.

Answer:

readme.txt

```

jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenge$ tcpdump -A -r tcpdump_challenge.pcap src 10.0.2.10 and dst 194.108.117.16 and port 21 | grep -i "RETR\|STOR"
reading from file tcpdump_challenge.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:54:21.713932 IP 10.0.2.10.40124 > 194.108.117.16.ftp: Flags [P.], seq 61:78, ack 560, win 65032, length 17: FTP: RETR readme.txt
.l....._. D.P...C...RETR readme.txt
jeffreyoo@jeffreyoo-VirtualBox:~/Desktop/SOC Course Files/02_Network_Security/01_tcpdump/Challenge$

```

Figure 9: Terminal output isolating the RETR command for the file "readme.txt".

9. Based on the unique User-Agent string found within the HTTP requests, what is the name of the related malware the endpoint might be infected with?

Command:

```

tcpdump -A -r tcpdump_challenge.pcap 'port 80' | grep
'User-Agent:' | sort -u

```

Command Breakdown:

- **tcpdump**: Invokes the packet analyzer.
- **-A**: Prints the payload of the packets in ASCII (human readable plain text). This is crucial for reading plain-text application layer data like HTTP methods.
- **-r tcpdump_challenge.pcap**: Reads the packets from the specified file instead of a live network interface.
- **port 80**: Filters for standard **HTTP** traffic.
- **grep 'User-Agent:'**: Isolates the specific HTTP header that identifies the client software.
- **sort -u**: Sorts the results and only prints unique (**-u**) entries, removing duplicates.

Terminal Return:

A list of User-Agent strings which are essentially standard browsers (e.g., Mozilla/5.0...) and one highly unusual or specific User-Agent string:

TeslaBrowser/5.5.

```

**
...C...P>Y.D. .-P.)xIx..GET /+zz0192lskaaa HTTP/1.1
Host: t.me
User-Agent: TeslaBrowser/5.5
Accept: */*

```

Figure 10: Detection of the "TeslaBrowser/5.5" User-Agent string.

Take that unique (suspicious) **User-Agent** string and perform an OSINT Google search. Malware families often have hardcoded, distinct User-Agents that cybersecurity researchers have documented. Search for the malware commonly associated with **TeslaBrowser/5.5**.

Answer:

Lumma Stealer (a.k.a LummaC2)

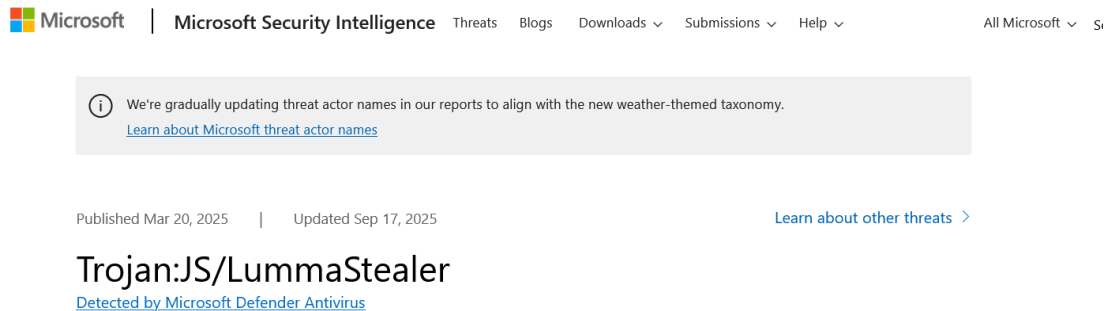


Figure 11: OSINT search results linking the User-Agent to Lumma Stealer.

LummaStealer then communicates with hardcoded domains, many using the .shop TLD like *futureddospzmvql.shop* and *writerospzm.shop*, which are often obfuscated and hidden behind Cloudflare proxies. Known C2 server IPs include 82.117.255.[127 and 77.73.134].68. A consistent indicator is the use of the unique user agent **TeslaBrowser/5.5** for these communications. If these primary C2s are unreachable, the malware has fallback mechanisms, including retrieving new domain instructions from pre-defined Telegram channels or even decoding them from specific Steam profile names.

Figure 12: Additional OSINT technical details regarding the malware family.

10. In defanged format, what was the full URL that the endpoint tried to connect to using the user agent identified above?

Command:

```
tcpdump -A -r tcpdump_challenge.pcap 'port 80' | grep -B 2 -A 5 'TeslaBrowser/5.5'
```

Command Breakdown:

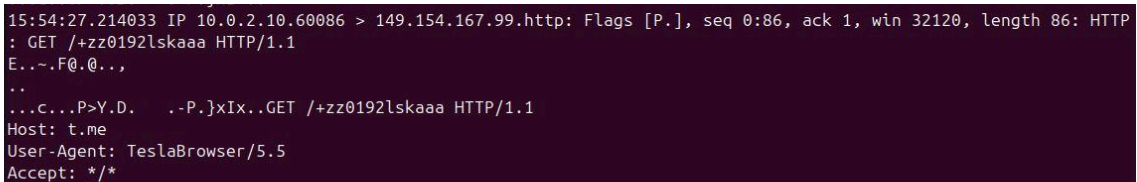
- **tcpdump**: Invokes the packet analyzer.
- **-A**: Prints the payload of the packets in ASCII (human readable plain text).
This is crucial for reading plain-text application layer data like HTTP methods.

- `-r tcpdump_challenge.pcap`: Reads the packets from the specified file instead of a live network interface.
- `port 80`: Filters for standard **HTTP** traffic.
- `-B 2 -A 5`: Tells `grep` to print 2 lines **B**efore the match, and 5 lines **A**fter the match. This is crucial because an HTTP request spans multiple lines. You need the `GET /path HTTP/1.1` line (which comes before the User-Agent) and the `Host: badguy.com` line (which usually comes after).

Terminal Return:

Read the output block to piece together the URL using the following:

1. `Host: t.me`
2. `GET /+zz0192lskaaa HTTP/1.1`



```
15:54:27.214033 IP 10.0.2.10.60086 > 149.154.167.99.http: Flags [P.], seq 0:86, ack 1, win 32120, length 86: HTTP
: GET /+zz0192lskaaa HTTP/1.1
E...F@.0..,
..
...C...P>Y.D.  .-P.}xIx..GET /+zz0192lskaaa HTTP/1.1
Host: t.me
User-Agent: TeslaBrowser/5.5
Accept: */*
```

Figure 13: Terminal output showing the Host and GET lines used to construct the malicious URL.

Combined to form the URL: `http://t.me/+zz0192lskaaa`

Defang the URL:

The URL was defanged using the security web tool, **CyberChef**.

Answer:

`hxxp[ :// ]t[ . ]me/+zz0192lskaaa`

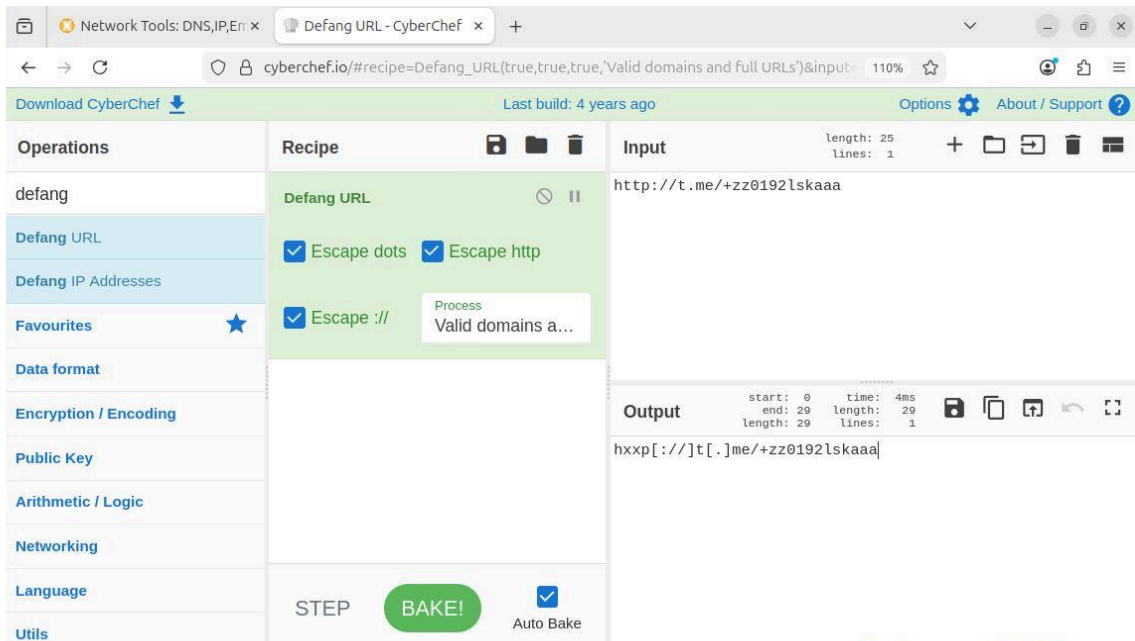


Figure 14: CyberChef tool used to defang the malicious URL.